やさしい PyTorch v1.0

初心者のための必須ガイド

著者*: jw*

この本は PyTorch の主要な関数をカテゴリ別に整理しています。各関数には以下が含まれます：

- コード例 - 実行可能な Python コード
- 実行結果 - 実際の出力結果
- 説明 - 簡潔明瞭な解説

PyTorch 2.8.0 ベース

目次

0. ハローワールド
1. テンソル作成
2. 基本演算
3. 形状操作
4. インデックスとスライス
5. リダクション操作
6. 数学関数
7. 線形代数
8. ニューラルネットワーク関数
9. 損失関数
10. プーリングと畳み込み
11. 高度な操作
12. 自動微分
13. デバイス操作
14. ユーティリティ
15. 比較操作
16. テンソルメソッド

ハローワールド

この章では、シンプルな例で完全なニューラルネットワークを紹介します。小さなニューラルネットワークに XOR 関数を教えてみましょう。

XOR とは？

入力 A	入力 B	出力 (A XOR B)
0	0	0
0	1	1
1	0	1
1	1	0

4 つのステップ

1. データ - 入力/出力ペアを定義
2. モデル - ニューラルネットワークを作成

3. 訓練 - データから学習（順伝播 → 損失 → 逆伝播 → 更新）
4. 推論 - 予測を行う

完全なコード

```
import torch
import torch.nn as nn

# === ハローワールド：超シンプルニューラルネットワーク ===
# 目標：XOR 関数を学習 (0^0=0, 0^1=1, 1^0=1, 1^1=0)

# 1. データ - 入力/出力ペア
X = torch.tensor([[0., 0.],
                  [0., 1.],
                  [1., 0.],
                  [1., 1.]])

y = torch.tensor([[0.],
                  [1.],
                  [1.],
                  [0.]])

print("=== データ ===")
print(f" 入力 X:\n{X}")
print(f" 正解 y: {y.flatten().tolist()}")

# 2. モデル - 2 層ニューラルネットワーク
model = nn.Sequential(
    nn.Linear(2, 4),   # 入力 2 個 -> 隠れ層 4 個
    nn.ReLU(),         # 活性化関数
    nn.Linear(4, 1),   # 隠れ層 4 個 -> 出力 1 個
    nn.Sigmoid()       # 出力 0~1
)

print(f"\n=== モデル ===")
print(model)

# 3. 訓練
criterion = nn.MSELoss()
optimizer = torch.optim.SGD(model.parameters(), lr=1.0)

print(f"\n=== 訓練 ===")
for epoch in range(1000):
    pred = model(X)
    loss = criterion(pred, y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    if epoch % 200 == 0:
        print(f" エポック {epoch:4d}, 損失: {loss.item():.4f}")

# 4. 推論
print(f"\n=== 推論 ===")
```

```python
with torch.no_grad():
    predictions = model(X)
    rounded = (predictions > 0.5).float()

print(" 入力 -> 予測 -> 丸め -> 正解")
for i in range(4):
    inp = X[i].tolist()
    pred_val = predictions[i].item()
    round_val = int(rounded[i].item())
    target = int(y[i].item())
    status = " 正解" if round_val == target else " 不正解"
    print(f"{inp} -> {pred_val:.3f} -> {round_val} -> {target} {status}")

accuracy = (rounded == y).float().mean()
print(f"\n精度: {accuracy.item()*100:.0f}%")
```

実行結果

```
=== データ ===
入力 X:
tensor([[0., 0.],
        [0., 1.],
        [1., 0.],
        [1., 1.]])
正解 y: [0.0, 1.0, 1.0, 0.0]

=== モデル ===
Sequential(
  (0): Linear(in_features=2, out_features=4, bias=True)
  (1): ReLU()
  (2): Linear(in_features=4, out_features=1, bias=True)
  (3): Sigmoid()
)

=== 訓練 ===
エポック    0, 損失: 0.2455
エポック  200, 損失: 0.1671
エポック  400, 損失: 0.1668
エポック  600, 損失: 0.1668
エポック  800, 損失: 0.1667

=== 推論 ===
入力 -> 予測 -> 丸め -> 正解
[0.0, 0.0] -> 0.334 -> 0 -> 0 正解
[0.0, 1.0] -> 0.988 -> 1 -> 1 正解
[1.0, 0.0] -> 0.334 -> 0 -> 1 不正解
[1.0, 1.0] -> 0.334 -> 0 -> 0 正解

精度: 75%
```

核心概念

- `nn.Sequential` - 層を順番に積み重ねる

- nn.Linear(in, out) - 全結合層
- nn.ReLU() - 活性化関数
- nn.MSELoss() - 損失関数（誤差の測定）
- optimizer.zero_grad() - 勾配を初期化
- loss.backward() - 勾配を計算（逆伝播）
- optimizer.step() - 重みを更新
- torch.no_grad() - 推論時は勾配を無効化

完了！初めてのニューラルネットワーク訓練が完了しました。

🔧 テンソル作成

torch.tensor(data) データ（リスト、*numpy* 配列など）からテンソルを作成。*dtype* を自動推論。

例:

```
import torch
# Create tensor from Python list
x = torch.tensor([[1, 2], [3, 4]])
print("Tensor from list:")
print(x)
print(f"Shape: {x.shape}, dtype: {x.dtype}")
```

実行結果:

```
Tensor from list:
tensor([[1, 2],
        [3, 4]])
Shape: torch.Size([2, 2]), dtype: torch.int64
```

torch.zeros(*size) ゼロで埋められたテンソルを作成。初期化に便利。

例:

```
import torch
# Create tensor filled with zeros
x = torch.zeros(2, 3)
print("Zeros tensor (2x3):")
print(x)
```

実行結果:

```
Zeros tensor (2x3):
tensor([[0., 0., 0.],
        [0., 0., 0.]])
```

torch.ones(*size) *1* で埋められたテンソルを作成。マスクや初期化によく使用。

例:

```
import torch
# Create tensor filled with ones
x = torch.ones(3, 2)
print("Ones tensor (3x2):")
print(x)
```

実行結果:

```
Ones tensor (3x2):
tensor([[1., 1.],
        [1., 1.],
        [1., 1.]])
```

torch.eye(n) # identity matrix 単位行列を作成 (対角線は *1*、他は *0*)。線形代数で使用。

例:

```python
import torch
# Create identity matrix
x = torch.eye(3)
print("Identity matrix (3x3):")
print(x)
```

実行結果:

```
Identity matrix (3x3):
tensor([[1., 0., 0.],
        [0., 1., 0.],
        [0., 0., 1.]])
```

torch.arange(start, end, step) 等間隔の *1D* テンソルを作成。*Python* の *range()* と同様。

例:

```python
import torch
# Create range tensor
x = torch.arange(0, 10, 2)
print("Range [0, 10) step 2:")
print(x)
```

実行結果:

```
Range [0, 10) step 2:
tensor([0, 2, 4, 6, 8])
```

torch.linspace(start, end, steps) 開始と終了の間に均等に分布した点のテンソルを作成。

例:

```python
import torch
# Create linearly spaced tensor
x = torch.linspace(0, 1, 5)
print("Linspace 0 to 1, 5 points:")
print(x)
```

実行結果:

```
Linspace 0 to 1, 5 points:
tensor([0.0000, 0.2500, 0.5000, 0.7500, 1.0000])
```

torch.logspace(start, end, steps) 対数スケールで均等分布したテンソルを作成。学習率設定に便利。

例:

```
import torch
# Create logarithmically spaced tensor
x = torch.logspace(0, 2, 3)  # 10^0, 10^1, 10^2
print("Logspace 10^0 to 10^2:")
print(x)
```

実行結果:

```
Logspace 10^0 to 10^2:
tensor([  1.,  10., 100.])
```

`torch.rand(*size)  # uniform [0,1)` *0* と *1* の間の一様分布の乱数テンソルを作成。

例:

```
import torch
torch.manual_seed(42)
# Create random tensor [0, 1)
x = torch.rand(2, 3)
print("Random uniform [0,1):")
print(x)
```

実行結果:

```
Random uniform [0,1):
tensor([[0.8823, 0.9150, 0.3829],
        [0.9593, 0.3904, 0.6009]])
```

`torch.randn(*size)  # normal N(0,1)` 標準正規分布（平均 *=0*、標準偏差 *=1*）からサンプリングしてテンソルを作成。

例:

```
import torch
torch.manual_seed(42)
# Create random tensor from normal distribution
x = torch.randn(2, 3)
print("Random normal N(0,1):")
print(x)
```

実行結果:

```
Random normal N(0,1):
tensor([[ 0.3367,  0.1288,  0.2345],
        [ 0.2303, -1.1229, -0.1863]])
```

`torch.randint(low, high, size)` *[low, high)* 範囲のランダム整数テンソルを作成。

例:

```
import torch
torch.manual_seed(42)
# Create random integers
x = torch.randint(0, 10, (2, 3))
print("Random integers [0, 10):")
print(x)
```

実行結果:

```
Random integers [0, 10):
tensor([[2, 7, 6],
        [4, 6, 5]])
```

torch.empty(*size) 未初期化のテンソルを作成。*zeros/ones* より高速だがゴミ値を含む。

例:

```python
import torch
# Create uninitialized tensor
x = torch.empty(2, 2)
print("Empty tensor (uninitialized):")
print(x)
print("Warning: Contains garbage values!")
```

実行結果:

```
Empty tensor (uninitialized):
tensor([[0., 0.],
        [0., 0.]])
Warning: Contains garbage values!
```

torch.full(size, fill_value) 特定の値で埋められたテンソルを作成。定数に便利。

例:

```python
import torch
# Create tensor filled with specific value
x = torch.full((2, 3), 7.0)
print("Tensor filled with 7.0:")
print(x)
```

実行結果:

```
Tensor filled with 7.0:
tensor([[7., 7., 7.],
        [7., 7., 7.]])
```

torch.zeros_like(x), ones_like(x) 入力テンソルと同じ形状と *dtype* の *0/1* テンソルを作成。

例:

```python
import torch
original = torch.tensor([[1.0, 2.0], [3.0, 4.0]])
zeros = torch.zeros_like(original)
ones = torch.ones_like(original)
print("Original:", original.shape)
print("Zeros like:", zeros)
print("Ones like:", ones)
```

実行結果:

```
Original: torch.Size([2, 2])
Zeros like: tensor([[0., 0.],
        [0., 0.]])
Ones like: tensor([[1., 1.],
        [1., 1.]])
```

基本演算

torch.add(a, b) or a + b 要素ごとの加算。異なる形状のブロードキャストをサポート。

例:

```
import torch
a = torch.tensor([1, 2, 3])
b = torch.tensor([4, 5, 6])
result = torch.add(a, b)
print(f"{a} + {b} = {result}")
```

実行結果:

```
tensor([1, 2, 3]) + tensor([4, 5, 6]) = tensor([5, 7, 9])
```

torch.sub(a, b) or a - b 要素ごとの減算。ブロードキャストをサポート。

例:

```
import torch
a = torch.tensor([5, 6, 7])
b = torch.tensor([1, 2, 3])
result = torch.sub(a, b)
print(f"{a} - {b} = {result}")
```

実行結果:

```
tensor([5, 6, 7]) - tensor([1, 2, 3]) = tensor([4, 4, 4])
```

torch.mul(a, b) or a * b 要素ごとの乗算 (アダマール積)。行列乗算ではない。

例:

```
import torch
a = torch.tensor([1, 2, 3])
b = torch.tensor([4, 5, 6])
result = torch.mul(a, b)
print(f"{a} * {b} = {result}")
```

実行結果:

```
tensor([1, 2, 3]) * tensor([4, 5, 6]) = tensor([ 4, 10, 18])
```

torch.div(a, b) or a / b 要素ごとの除算。整数除算を避けるため *float* テンソルを使用。

例:

```
import torch
a = torch.tensor([10.0, 20.0, 30.0])
b = torch.tensor([2.0, 4.0, 5.0])
result = torch.div(a, b)
print(f"{a} / {b} = {result}")
```

実行結果:

```
tensor([10., 20., 30.]) / tensor([2., 4., 5.]) = tensor([5., 5., 6.])
```

torch.matmul(a, b) or a @ b　行列乗算。*2D* テンソルの場合、標準的な行列積を実行。

例:

```
import torch
a = torch.tensor([[1, 2], [3, 4]])
b = torch.tensor([[5, 6], [7, 8]])
result = torch.matmul(a, b)
print("Matrix A:")
print(a)
print("Matrix B:")
print(b)
print("A @ B:")
print(result)
```

実行結果:

```
Matrix A:
tensor([[1, 2],
        [3, 4]])
Matrix B:
tensor([[5, 6],
        [7, 8]])
A @ B:
tensor([[19, 22],
        [43, 50]])
```

torch.pow(a, exp) or a ** exp　要素ごとの累乗演算。スカラーまたはテンソル指数を使用可能。

例:

```
import torch
x = torch.tensor([1.0, 2.0, 3.0])
result = torch.pow(x, 2)
print(f"{x} ** 2 = {result}")
```

実行結果:

```
tensor([1., 2., 3.]) ** 2 = tensor([1., 4., 9.])
```

torch.abs(x) # absolute value　各要素の絶対値を返す。

例:

```
import torch
x = torch.tensor([-1, -2, 3, -4])
result = torch.abs(x)
print(f"abs({x}) = {result}")
```

実行結果:

```
abs(tensor([-1, -2,  3, -4])) = tensor([1, 2, 3, 4])
```

torch.neg(x) # negative　各要素の負数を返す。*-x* と同じ。

例:

```
import torch
x = torch.tensor([1, -2, 3])
result = torch.neg(x)
print(f"neg({x}) = {result}")
```

実行結果:

```
neg(tensor([ 1, -2,  3])) = tensor([-1,  2, -3])
```

`torch.reciprocal(x)  # 1/x` 各要素の逆数 (*1/x*) を返す。

例:

```
import torch
x = torch.tensor([1.0, 2.0, 4.0])
result = torch.reciprocal(x)
print(f"1/{x} = {result}")
```

実行結果:

```
1/tensor([1., 2., 4.]) = tensor([1.0000, 0.5000, 0.2500])
```

`torch.remainder(a, b)  # remainder` 要素ごとの剰余 (モジュロ演算)。

例:

```
import torch
a = torch.tensor([10, 11, 12])
b = torch.tensor([3, 3, 3])
result = torch.remainder(a, b)
print(f"{a} % {b} = {result}")
```

実行結果:

```
tensor([10, 11, 12]) % tensor([3, 3, 3]) = tensor([1, 2, 0])
```

形状操作

`x.reshape(*shape)` 新しい形状のテンソルを返す。要素総数は同じ必要あり。データをコピーする場合あり。

例:

```
import torch
x = torch.arange(6)
print(f"Original: {x}")
reshaped = x.reshape(2, 3)
print("Reshaped to (2, 3):")
print(reshaped)
```

実行結果:

```
Original: tensor([0, 1, 2, 3, 4, 5])
Reshaped to (2, 3):
tensor([[0, 1, 2],
        [3, 4, 5]])
```

x.view(*shape) 新しい形状のビューを返す。連続メモリが必要。データを共有。

例:

```
import torch
x = torch.arange(12)
print(f"Original: {x}")
viewed = x.view(3, 4)
print("View as (3, 4):")
print(viewed)
```

実行結果:

```
Original: tensor([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11])
View as (3, 4):
tensor([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]])
```

x.transpose(dim0, dim1) *2* つの次元を交換。*2D* の場合は行列転置と同じ。

例:

```
import torch
x = torch.tensor([[1, 2, 3], [4, 5, 6]])
print("Original (2x3):")
print(x)
transposed = x.transpose(0, 1)
print("Transposed (3x2):")
print(transposed)
```

実行結果:

```
Original (2x3):
tensor([[1, 2, 3],
        [4, 5, 6]])
Transposed (3x2):
tensor([[1, 4],
        [2, 5],
        [3, 6]])
```

x.permute(*dims) すべての次元を再配置。*transpose* より柔軟。

例:

```
import torch
x = torch.randn(2, 3, 4)
print(f"Original shape: {x.shape}")
permuted = x.permute(2, 0, 1)
print(f"Permuted shape: {permuted.shape}")
```

実行結果:

```
Original shape: torch.Size([2, 3, 4])
Permuted shape: torch.Size([4, 2, 3])
```

x.squeeze(dim) サイズが *1* の次元を削除。テンソルのランクを減らす。

例:

```
import torch
x = torch.zeros(1, 3, 1, 4)
print(f"Original shape: {x.shape}")
squeezed = x.squeeze()
print(f"Squeezed shape: {squeezed.shape}")
```

実行結果:

```
Original shape: torch.Size([1, 3, 1, 4])
Squeezed shape: torch.Size([3, 4])
```

x.unsqueeze(dim) 指定位置にサイズ *1* の次元を追加。

例:

```
import torch
x = torch.tensor([1, 2, 3])
print(f"Original shape: {x.shape}")
unsqueezed = x.unsqueeze(0)
print(f"Unsqueezed at dim 0: {unsqueezed.shape}")
print(unsqueezed)
```

実行結果:

```
Original shape: torch.Size([3])
Unsqueezed at dim 0: torch.Size([1, 3])
tensor([[1, 2, 3]])
```

x.flatten(start_dim, end_dim) テンソルを *1D* に平坦化。部分平坦化用のオプション開始/終了次元あり。

例:

```
import torch
x = torch.randn(2, 3, 4)
print(f"Original shape: {x.shape}")
flat = x.flatten()
print(f"Flattened: {flat.shape}")
```

実行結果:

```
Original shape: torch.Size([2, 3, 4])
Flattened: torch.Size([24])
```

x.expand(*sizes) サイズ *1* の次元に沿って繰り返してテンソルを拡張。データをコピーしない。

例:

```
import torch
x = torch.tensor([[1], [2], [3]])
print(f"Original: {x.shape}")
expanded = x.expand(3, 4)
print("Expanded to (3, 4):")
print(expanded)
```

実行結果:

```
Original: torch.Size([3, 1])
Expanded to (3, 4):
tensor([[1, 1, 1, 1],
        [2, 2, 2, 2],
        [3, 3, 3, 3]])
```

x.repeat(*sizes) 各次元に沿ってテンソルを繰り返す。新しいメモリを作成。

例:

```
import torch
x = torch.tensor([1, 2])
print(f"Original: {x}")
repeated = x.repeat(3)
print(f"Repeated 3x: {repeated}")
```

実行結果:

```
Original: tensor([1, 2])
Repeated 3x: tensor([1, 2, 1, 2, 1, 2])
```

x.contiguous() メモリ上で連続したテンソルを返す。*transpose* 後 *view()* 前に必要。

例:

```
import torch
x = torch.randn(3, 4)
y = x.transpose(0, 1)
print(f"Is contiguous: {y.is_contiguous()}")
z = y.contiguous()
print(f"After contiguous(): {z.is_contiguous()}")
```

実行結果:

```
Is contiguous: False
After contiguous(): True
```

□ インデックスとスライス

x[i] # index 基本インデックス、インデックス *i* の行/要素を返す。

例:

```
import torch
x = torch.tensor([[1, 2, 3], [4, 5, 6]])
print("Tensor:")
print(x)
print(f"x[0] = {x[0]}")
print(f"x[1] = {x[1]}")
```

実行結果:

```
Tensor:
tensor([[1, 2, 3],
        [4, 5, 6]])
x[0] = tensor([1, 2, 3])
x[1] = tensor([4, 5, 6])
```

`x[i:j]  # slice` *start:stop:step* でスライス。*Python* リストと同様。

例:

```
import torch
x = torch.arange(10)
print(f"Original: {x}")
print(f"x[2:5] = {x[2:5]}")
print(f"x[::2] = {x[::2]}")
```

実行結果:

```
Original: tensor([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
x[2:5] = tensor([2, 3, 4])
x[::2] = tensor([0, 2, 4, 6, 8])
```

`x[..., i]  # ellipsis` 省略記号 *(...)* は残りのすべての次元を表す。

例:

```
import torch
x = torch.randn(2, 3, 4)
print(f"Shape: {x.shape}")
print(f"x[..., 0] shape: {x[..., 0].shape}")
print(f"x[0, ...] shape: {x[0, ...].shape}")
```

実行結果:

```
Shape: torch.Size([2, 3, 4])
x[..., 0] shape: torch.Size([2, 3])
x[0, ...] shape: torch.Size([3, 4])
```

`x[:, -1]  # last column` 負のインデックスは末尾から。*-1* は最後の要素。

例:

```
import torch
x = torch.tensor([[1, 2, 3], [4, 5, 6]])
print("Tensor:")
print(x)
print(f"Last column x[:, -1] = {x[:, -1]}")
```

実行結果:

```
Tensor:
tensor([[1, 2, 3],
        [4, 5, 6]])
Last column x[:, -1] = tensor([3, 6])
```

`torch.index_select(x, dim, idx)` インデックステンソルを使って次元に沿って要素を選択。

例:

```
import torch
x = torch.tensor([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
idx = torch.tensor([0, 2])
result = torch.index_select(x, 0, idx)
print("Original:")
print(x)
```

```
print(f"Select rows {idx.tolist()}:")
print(result)
```

実行結果:

```
Original:
tensor([[1, 2, 3],
        [4, 5, 6],
        [7, 8, 9]])
Select rows [0, 2]:
tensor([[1, 2, 3],
        [7, 8, 9]])
```

`torch.masked_select(x, mask)` マスクが *True* の要素の *1D* テンソルを返す。

例:

```
import torch
x = torch.tensor([[1, 2], [3, 4]])
mask = x > 2
print(f"Tensor: {x}")
print(f"Mask (>2): {mask}")
print(f"Selected: {torch.masked_select(x, mask)}")
```

実行結果:

```
Tensor: tensor([[1, 2],
        [3, 4]])
Mask (>2): tensor([[False, False],
        [ True,  True]])
Selected: tensor([3, 4])
```

`torch.gather(x, dim, idx)  # gather` インデックスに従って軸に沿って値を収集。分布から選択に便利。

例:

```
import torch
x = torch.tensor([[1, 2], [3, 4]])
idx = torch.tensor([[0, 0], [1, 0]])
result = torch.gather(x, 1, idx)
print("Original:")
print(x)
print(f"Gather with indices {idx.tolist()}:")
print(result)
```

実行結果:

```
Original:
tensor([[1, 2],
        [3, 4]])
Gather with indices [[0, 0], [1, 0]]:
tensor([[1, 1],
        [4, 3]])
```

torch.scatter(x, dim, idx, src) *src* の値を *x* の *idx* 指定位置に書き込む。*gather* の逆操作。

例:

```
import torch
x = torch.zeros(3, 5)
idx = torch.tensor([[0, 1, 2, 0, 0], [2, 0, 0, 1, 2]])
src = torch.ones(2, 5)
result = x.scatter(0, idx, src)
print("Scatter result:")
print(result)
```

実行結果:

```
Scatter result:
tensor([[1., 1., 1., 1., 1.],
        [0., 1., 0., 1., 0.],
        [1., 0., 1., 0., 1.]])
```

torch.where(cond, x, y) # conditional 条件が *True* の場所で *x* の要素を、そうでなければ *y* の要素を返す。

例:

```
import torch
x = torch.tensor([1, 2, 3, 4, 5])
y = torch.tensor([10, 20, 30, 40, 50])
cond = x > 3
result = torch.where(cond, x, y)
print(f"x: {x}")
print(f"y: {y}")
print(f"where(x>3, x, y): {result}")
```

実行結果:

```
x: tensor([1, 2, 3, 4, 5])
y: tensor([10, 20, 30, 40, 50])
where(x>3, x, y): tensor([10, 20, 30,  4,  5])
```

torch.take(x, indices) # flat index テンソルを *1D* として扱い、指定インデックスの要素を返す。

例:

```
import torch
x = torch.tensor([[1, 2, 3], [4, 5, 6]])
idx = torch.tensor([0, 2, 5])
result = torch.take(x, idx)
print(f"Tensor (flattened would be {x.flatten().tolist()})")
print(f"Take indices {idx.tolist()}: {result}")
```

実行結果:

```
Tensor (flattened would be [1, 2, 3, 4, 5, 6])
Take indices [0, 2, 5]: tensor([1, 3, 6])
```

Σ リダクション操作

x.sum(dim, keepdim) 要素を合計。オプションの *dim* で軸を指定。*keepdim* で次元を維持。

例:

```
import torch
x = torch.tensor([[1, 2, 3], [4, 5, 6]])
print("Tensor:")
print(x)
print(f"Sum all: {x.sum()}")
print(f"Sum dim=0: {x.sum(dim=0)}")
print(f"Sum dim=1: {x.sum(dim=1)}")
```

実行結果:

```
Tensor:
tensor([[1, 2, 3],
        [4, 5, 6]])
Sum all: 21
Sum dim=0: tensor([5, 7, 9])
Sum dim=1: tensor([ 6, 15])
```

x.mean(dim, keepdim) 平均を計算。*float* テンソルが必要。

例:

```
import torch
x = torch.tensor([[1.0, 2.0], [3.0, 4.0]])
print("Tensor:")
print(x)
print(f"Mean all: {x.mean()}")
print(f"Mean dim=1: {x.mean(dim=1)}")
```

実行結果:

```
Tensor:
tensor([[1., 2.],
        [3., 4.]])
Mean all: 2.5
Mean dim=1: tensor([1.5000, 3.5000])
```

x.std(dim, unbiased) 標準偏差を計算。*unbiased=True* は *N-1* を分母に使用。

例:

```
import torch
x = torch.tensor([1.0, 2.0, 3.0, 4.0, 5.0])
print(f"Data: {x}")
print(f"Std (unbiased): {x.std():.4f}")
print(f"Std (biased): {x.std(unbiased=False):.4f}")
```

実行結果:

```
Data: tensor([1., 2., 3., 4., 5.])
Std (unbiased): 1.5811
Std (biased): 1.4142
```

x.var(dim, unbiased) 分散（標準偏差の二乗）を計算。

例:

```
import torch
x = torch.tensor([1.0, 2.0, 3.0, 4.0, 5.0])
print(f"Data: {x}")
print(f"Variance: {x.var():.4f}")
```

実行結果:

```
Data: tensor([1., 2., 3., 4., 5.])
Variance: 2.5000
```

x.max(dim) # values & indices 次元に沿って最大値とインデックスを返す。

例:

```
import torch
x = torch.tensor([[1, 5, 3], [4, 2, 6]])
print("Tensor:")
print(x)
vals, idxs = x.max(dim=1)
print(f"Max per row: values={vals}, indices={idxs}")
```

実行結果:

```
Tensor:
tensor([[1, 5, 3],
        [4, 2, 6]])
Max per row: values=tensor([5, 6]), indices=tensor([1, 2])
```

x.min(dim) # values & indices 次元に沿って最小値とインデックスを返す。

例:

```
import torch
x = torch.tensor([[1, 5, 3], [4, 2, 6]])
print("Tensor:")
print(x)
vals, idxs = x.min(dim=1)
print(f"Min per row: values={vals}, indices={idxs}")
```

実行結果:

```
Tensor:
tensor([[1, 5, 3],
        [4, 2, 6]])
Min per row: values=tensor([1, 2]), indices=tensor([0, 1])
```

x.argmax(dim) # indices only 最大値のインデックスを返す。*softmax* 出力とよく使用。

例:

```
import torch
x = torch.tensor([[1, 5, 3], [4, 2, 6]])
print("Tensor:")
print(x)
```

```
print(f"Argmax (all): {x.argmax()}")
print(f"Argmax dim=1: {x.argmax(dim=1)}")
```

実行結果:

```
Tensor:
tensor([[1, 5, 3],
        [4, 2, 6]])
Argmax (all): 5
Argmax dim=1: tensor([1, 2])
```

x.argmin(dim) # indices only 最小値のインデックスを返す。

例:

```
import torch
x = torch.tensor([[1, 5, 3], [4, 2, 6]])
print("Tensor:")
print(x)
print(f"Argmin dim=1: {x.argmin(dim=1)}")
```

実行結果:

```
Tensor:
tensor([[1, 5, 3],
        [4, 2, 6]])
Argmin dim=1: tensor([0, 1])
```

x.median(dim) 次元に沿って中央値とインデックスを返す。

例:

```
import torch
x = torch.tensor([[1, 5, 3], [4, 2, 6]])
print("Tensor:")
print(x)
vals, idxs = x.median(dim=1)
print(f"Median per row: values={vals}, indices={idxs}")
```

実行結果:

```
Tensor:
tensor([[1, 5, 3],
        [4, 2, 6]])
Median per row: values=tensor([3, 4]), indices=tensor([2, 0])
```

x.mode(dim) 次元に沿って最頻値を返す。

例:

```
import torch
x = torch.tensor([[1, 1, 2], [3, 3, 3]])
print("Tensor:")
print(x)
vals, idxs = x.mode(dim=1)
print(f"Mode per row: values={vals}")
```

実行結果:

```
Tensor:
tensor([[1, 1, 2],
        [3, 3, 3]])
Mode per row: values=tensor([1, 3])
```

x.prod(dim) # product 次元に沿って要素の積を計算。

例:

```
import torch
x = torch.tensor([[1, 2, 3], [4, 5, 6]])
print("Tensor:")
print(x)
print(f"Product all: {x.prod()}")
print(f"Product dim=1: {x.prod(dim=1)}")
```

実行結果:

```
Tensor:
tensor([[1, 2, 3],
        [4, 5, 6]])
Product all: 720
Product dim=1: tensor([  6, 120])
```

x.cumsum(dim) # cumulative sum 次元に沿って累積和を計算。各要素は以前のすべての要素の和。

例:

```
import torch
x = torch.tensor([1, 2, 3, 4])
print(f"Original: {x}")
print(f"Cumsum: {x.cumsum(dim=0)}")
```

実行結果:

```
Original: tensor([1, 2, 3, 4])
Cumsum: tensor([ 1,  3,  6, 10])
```

x.norm(p, dim) # Lp norm *Lp* ノルムを計算。*L2* (ユークリッド) がデフォルト。*L1* はマンハッタン距離。

例:

```
import torch
x = torch.tensor([3.0, 4.0])
print(f"Vector: {x}")
print(f"L2 norm: {x.norm():.4f}")  # sqrt(9+16) = 5
print(f"L1 norm: {x.norm(p=1):.4f}")  # 3+4 = 7
```

実行結果:

```
Vector: tensor([3., 4.])
L2 norm: 5.0000
L1 norm: 7.0000
```

∫ 数学関数

torch.sin(x), cos(x), tan(x)　三角関数。入力はラジアン。

例:

```
import torch
import math
x = torch.tensor([0, math.pi/2, math.pi])
print(f"x: {x}")
print(f"sin(x): {torch.sin(x)}")
print(f"cos(x): {torch.cos(x)}")
```

実行結果:

```
x: tensor([0.0000, 1.5708, 3.1416])
sin(x): tensor([ 0.0000e+00,  1.0000e+00, -8.7423e-08])
cos(x): tensor([ 1.0000e+00, -4.3711e-08, -1.0000e+00])
```

torch.asin(x), acos(x), atan(x)　逆三角関数。ラジアンを返す。

例:

```
import torch
x = torch.tensor([0.0, 0.5, 1.0])
print(f"x: {x}")
print(f"asin(x): {torch.asin(x)}")
print(f"acos(x): {torch.acos(x)}")
```

実行結果:

```
x: tensor([0.0000, 0.5000, 1.0000])
asin(x): tensor([0.0000, 0.5236, 1.5708])
acos(x): tensor([1.5708, 1.0472, 0.0000])
```

torch.sinh(x), cosh(x), tanh(x)　双曲線関数。*tanh* は活性化関数としてよく使用。

例:

```
import torch
x = torch.tensor([-1.0, 0.0, 1.0])
print(f"x: {x}")
print(f"tanh(x): {torch.tanh(x)}")
```

実行結果:

```
x: tensor([-1.,  0.,  1.])
tanh(x): tensor([-0.7616,  0.0000,  0.7616])
```

torch.exp(x), log(x), log10(x)　指数と対数関数。*log* は自然対数（底 *e*）。

例:

```
import torch
x = torch.tensor([1.0, 2.0, 3.0])
print(f"x: {x}")
print(f"exp(x): {torch.exp(x)}")
print(f"log(exp(x)): {torch.log(torch.exp(x))}")
```

実行結果:

```
x: tensor([1., 2., 3.])
exp(x): tensor([ 2.7183,  7.3891, 20.0855])
log(exp(x)): tensor([1.0000, 2.0000, 3.0000])
```

`torch.sqrt(x)`, `rsqrt(x)` 平方根と逆平方根 (*1/sqrt(x)*)。

例:

```
import torch
x = torch.tensor([1.0, 4.0, 9.0])
print(f"x: {x}")
print(f"sqrt(x): {torch.sqrt(x)}")
print(f"rsqrt(x): {torch.rsqrt(x)}")  # 1/sqrt(x)
```

実行結果:

```
x: tensor([1., 4., 9.])
sqrt(x): tensor([1., 2., 3.])
rsqrt(x): tensor([1.0000, 0.5000, 0.3333])
```

`torch.floor(x)`, `ceil(x)`, `round(x)` 丸め関数。*floor* は切り捨て、*ceil* は切り上げ。

例:

```
import torch
x = torch.tensor([1.2, 2.5, 3.7])
print(f"x: {x}")
print(f"floor(x): {torch.floor(x)}")
print(f"ceil(x): {torch.ceil(x)}")
print(f"round(x): {torch.round(x)}")
```

実行結果:

```
x: tensor([1.2000, 2.5000, 3.7000])
floor(x): tensor([1., 2., 3.])
ceil(x): tensor([2., 3., 4.])
round(x): tensor([1., 2., 4.])
```

`torch.clamp(x, min, max)` 値を *[min, max]* 範囲に制限。範囲外の値は境界値に設定。

例:

```
import torch
x = torch.tensor([-2, 0, 3, 5, 10])
result = torch.clamp(x, min=0, max=5)
print(f"Original: {x}")
print(f"Clamped [0,5]: {result}")
```

実行結果:

```
Original: tensor([-2,  0,  3,  5, 10])
Clamped [0,5]: tensor([0, 0, 3, 5, 5])
```

`torch.sign(x)` 各要素の符号に基づいて*-1*、*0*、または *1* を返す。

例:

```
import torch
x = torch.tensor([-3, 0, 5])
print(f"x: {x}")
print(f"sign(x): {torch.sign(x)}")
```

実行結果:

```
x: tensor([-3,  0,  5])
sign(x): tensor([-1,  0,  1])
```

`torch.sigmoid(x)` シグモイド関数：*1/(1+e^-x)*。値を *(0, 1)* にマッピング。二値分類に使用。

例:

```
import torch
x = torch.tensor([-2.0, 0.0, 2.0])
print(f"x: {x}")
print(f"sigmoid(x): {torch.sigmoid(x)}")
```

実行結果:

```
x: tensor([-2.,  0.,  2.])
sigmoid(x): tensor([0.1192, 0.5000, 0.8808])
```

線形代数

`torch.mm(a, b)  # 2D matrix mult` *2D* テンソルの行列乗算。バッチや高次元には *matmul* を使用。

例:

```
import torch
a = torch.tensor([[1, 2], [3, 4]])
b = torch.tensor([[5, 6], [7, 8]])
result = torch.mm(a, b)
print("A @ B =")
print(result)
```

実行結果:

```
A @ B =
tensor([[19, 22],
        [43, 50]])
```

`torch.bmm(a, b)  # batch mm` バッチ行列乗算。最初の次元がバッチサイズ。

例:

```
import torch
a = torch.randn(10, 3, 4)  # batch of 10 matrices
b = torch.randn(10, 4, 5)
result = torch.bmm(a, b)
print(f"Batch shapes: {a.shape} @ {b.shape} = {result.shape}")
```

実行結果:

```
Batch shapes: torch.Size([10, 3, 4]) @ torch.Size([10, 4, 5]) = torch.Size([10, 3, 5])
```

torch.mv(mat, vec) # matrix-vector 行列-ベクトル乗算。*vec* は列ベクトルとして扱う。

例:

```
import torch
mat = torch.tensor([[1, 2], [3, 4]])
vec = torch.tensor([1, 1])
result = torch.mv(mat, vec)
print(f"Matrix @ vector = {result}")
```

実行結果:

```
Matrix @ vector = tensor([3, 7])
```

torch.dot(a, b) # 1D dot product *1D* テンソルの内積。要素ごとの積の和。

例:

```
import torch
a = torch.tensor([1.0, 2.0, 3.0])
b = torch.tensor([4.0, 5.0, 6.0])
result = torch.dot(a, b)
print(f"{a} . {b} = {result}")
```

実行結果:

```
tensor([1., 2., 3.]) . tensor([4., 5., 6.]) = 32.0
```

torch.det(x) # determinant 正方行列の行列式を計算。

例:

```
import torch
x = torch.tensor([[1.0, 2.0], [3.0, 4.0]])
det = torch.det(x)
print("Matrix:")
print(x)
print(f"Determinant: {det:.4f}")
```

実行結果:

```
Matrix:
tensor([[1., 2.],
        [3., 4.]])
Determinant: -2.0000
```

torch.inverse(x) # matrix inverse 行列の逆行列を計算。*A @ A^-1* = 単位行列。

例:

```
import torch
x = torch.tensor([[1.0, 2.0], [3.0, 4.0]])
inv = torch.inverse(x)
print("Original:")
print(x)
print("Inverse:")
print(inv)
print("X @ X^-1:")
print(torch.mm(x, inv))
```

実行結果:

```
Original:
tensor([[1., 2.],
        [3., 4.]])
Inverse:
tensor([[-2.0000,  1.0000],
        [ 1.5000, -0.5000]])
X @ X^-1:
tensor([[1., 0.],
        [0., 1.]])
```

`torch.svd(x)  # singular value decomp` 特異値分解。*X = U @ diag(S) @ V^T*

例:

```
import torch
x = torch.tensor([[1.0, 2.0], [3.0, 4.0], [5.0, 6.0]])
U, S, V = torch.svd(x)
print(f"Original shape: {x.shape}")
print(f"U: {U.shape}, S: {S.shape}, V: {V.shape}")
print(f"Singular values: {S}")
```

実行結果:

```
Original shape: torch.Size([3, 2])
U: torch.Size([3, 2]), S: torch.Size([2]), V: torch.Size([2, 2])
Singular values: tensor([9.5255, 0.5143])
```

`torch.eig(x)  # eigenvalues` 固有値を計算。一般には *linalg.eig*、対称には *linalg.eigvalsh* を使用。

例:

```
import torch
x = torch.tensor([[1.0, 2.0], [2.0, 1.0]])
eigenvalues = torch.linalg.eigvalsh(x)  # For symmetric matrices
print("Matrix:")
print(x)
print(f"Eigenvalues: {eigenvalues}")
```

実行結果:

```
Matrix:
tensor([[1., 2.],
        [2., 1.]])
Eigenvalues: tensor([-1.,  3.])
```

`torch.linalg.norm(x, ord)` 行列またはベクトルノルムを計算。*Frobenius* が行列のデフォルト。

例:

```
import torch
x = torch.tensor([[1.0, 2.0], [3.0, 4.0]])
print("Matrix:")
print(x)
print(f"Frobenius norm: {torch.linalg.norm(x):.4f}")
print(f"L1 norm: {torch.linalg.norm(x, ord=1):.4f}")
```

実行結果:

```
Matrix:
tensor([[1., 2.],
        [3., 4.]])
Frobenius norm: 5.4772
L1 norm: 6.0000
```

`torch.linalg.solve(A, b)` 線形方程式系 $Ax = b$ を解く。逆行列計算より安定。

例:

```
import torch
A = torch.tensor([[3.0, 1.0], [1.0, 2.0]])
b = torch.tensor([9.0, 8.0])
x = torch.linalg.solve(A, b)
print(f"Solving Ax = b")
print(f"x = {x}")
print(f"Verify A@x = {A @ x}")
```

実行結果:

```
Solving Ax = b
x = tensor([2., 3.])
Verify A@x = tensor([9., 8.])
```

`torch.trace(x)  # sum of diagonal` 対角要素の和。単位行列の *trace* は次元数。

例:

```
import torch
x = torch.tensor([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print("Matrix:")
print(x)
print(f"Trace: {torch.trace(x)}")  # 1+5+9 = 15
```

実行結果:

```
Matrix:
tensor([[1, 2, 3],
        [4, 5, 6],
        [7, 8, 9]])
Trace: 15
```

`torch.outer(a, b)  # outer product` *2* つのベクトルの外積。結果の形状は *(len(a), len(b))*。

例:

```
import torch
a = torch.tensor([1, 2, 3])
b = torch.tensor([4, 5])
result = torch.outer(a, b)
print(f"a: {a}, b: {b}")
print("Outer product:")
print(result)
```

実行結果:

```
a: tensor([1, 2, 3]), b: tensor([4, 5])
Outer product:
tensor([[ 4,  5],
        [ 8, 10],
        [12, 15]])
```

◻ ニューラルネットワーク関数

活性化関数はニューラルネットワークに非線形性を追加。

`F.relu(x)  # max(0, x)` *ReLU* (整流線形ユニット)。*max(0, x)* を返す。最も一般的な活性化関数。

例:

```
import torch
import torch.nn.functional as F
x = torch.tensor([-2.0, -1.0, 0.0, 1.0, 2.0])
print(f"Input: {x}")
print(f"ReLU:  {F.relu(x)}")
```

実行結果:

```
Input: tensor([-2., -1.,  0.,  1.,  2.])
ReLU:  tensor([0., 0., 0., 1., 2.])
```

`F.leaky_relu(x, neg_slope)` *ReLU* に似ているが小さな負の値を許容。*'dying ReLU'* 問題を防止。

例:

```
import torch
import torch.nn.functional as F
x = torch.tensor([-2.0, -1.0, 0.0, 1.0, 2.0])
print(f"Input: {x}")
print(f"Leaky ReLU: {F.leaky_relu(x, 0.1)}")
```

実行結果:

```
Input: tensor([-2., -1.,  0.,  1.,  2.])
Leaky ReLU: tensor([-0.2000, -0.1000,  0.0000,  1.0000,  2.0000])
```

`F.gelu(x)  # Gaussian Error` ガウス誤差線形ユニット。*Transformer* (*BERT*、*GPT*) で使用。

例:

```
import torch
import torch.nn.functional as F
x = torch.tensor([-2.0, -1.0, 0.0, 1.0, 2.0])
print(f"Input: {x}")
print(f"GELU:  {F.gelu(x)}")
```

実行結果:

```
Input: tensor([-2., -1.,  0.,  1.,  2.])
GELU:  tensor([-0.0455, -0.1587,  0.0000,  0.8413,  1.9545])
```

`F.sigmoid(x)  # 1/(1+e^-x)` 値を *(0, 1)* にマッピング。二値分類出力に使用。

例:

```
import torch
import torch.nn.functional as F
x = torch.tensor([-2.0, 0.0, 2.0])
print(f"Input: {x}")
print(f"Sigmoid: {F.sigmoid(x)}")
```

実行結果:

```
Input: tensor([-2.,  0.,  2.])
Sigmoid: tensor([0.1192, 0.5000, 0.8808])
```

`F.tanh(x)  # hyperbolic tan` 値を *(-1, 1)* にマッピング。*0* 中心なので隠れ層で *sigmoid* より良い。

例:

```
import torch
import torch.nn.functional as F
x = torch.tensor([-2.0, 0.0, 2.0])
print(f"Input: {x}")
print(f"Tanh: {F.tanh(x)}")
```

実行結果:

```
Input: tensor([-2.,  0.,  2.])
Tanh: tensor([-0.9640,  0.0000,  0.9640])
```

`F.softmax(x, dim)  # probabilities` ロジットを確率 (合計 *1*) に変換。多クラス出力に使用。

例:

```
import torch
import torch.nn.functional as F
logits = torch.tensor([2.0, 1.0, 0.1])
probs = F.softmax(logits, dim=0)
print(f"Logits: {logits}")
print(f"Softmax: {probs}")
print(f"Sum: {probs.sum():.4f}")
```

実行結果:

```
Logits: tensor([2.0000, 1.0000, 0.1000])
Softmax: tensor([0.6590, 0.2424, 0.0986])
Sum: 1.0000
```

`F.log_softmax(x, dim)` *softmax* の対数。数値的により安定。*NLLLoss* と一緒に使用。

例:

```
import torch
import torch.nn.functional as F
logits = torch.tensor([2.0, 1.0, 0.1])
log_probs = F.log_softmax(logits, dim=0)
print(f"Logits: {logits}")
print(f"Log softmax: {log_probs}")
```

実行結果:

```
Logits: tensor([2.0000, 1.0000, 0.1000])
Log softmax: tensor([-0.4170, -1.4170, -2.3170])
```

正則化技術は訓練中の過学習を防止。

F.dropout(x, p, training) 確率 p で要素をランダムに 0 に設定。訓練時のみ有効。

例:

```
import torch
import torch.nn.functional as F
torch.manual_seed(42)
x = torch.ones(10)
dropped = F.dropout(x, p=0.5, training=True)
print(f"Original: {x}")
print(f"Dropout (p=0.5): {dropped}")
```

実行結果:

```
Original: tensor([1., 1., 1., 1., 1., 1., 1., 1., 1., 1.])
Dropout (p=0.5): tensor([2., 2., 2., 2., 0., 2., 0., 0., 2., 2.])
```

F.batch_norm(x, ...) # normalize バッチ次元に沿って正規化。訓練を安定化。

例:

```
import torch
import torch.nn.functional as F
x = torch.randn(2, 3, 4, 4)  # batch, channels, H, W
mean = torch.zeros(3)
var = torch.ones(3)
result = F.batch_norm(x, mean, var)
print(f"Input shape: {x.shape}")
print(f"After batch_norm: mean~{result.mean():.4f}, std~{result.std():.4f}")
```

実行結果:

```
Input shape: torch.Size([2, 3, 4, 4])
After batch_norm: mean~0.1266, std~0.9259
```

F.layer_norm(x, shape) 指定次元に沿って正規化。*Transformer* で使用。

例:

```
import torch
import torch.nn.functional as F
x = torch.randn(2, 3, 4)
result = F.layer_norm(x, [3, 4])
print(f"Input shape: {x.shape}")
print(f"After layer_norm: mean~{result.mean():.4f}, std~{result.std():.4f}")
```

実行結果:

```
Input shape: torch.Size([2, 3, 4])
After layer_norm: mean~0.0000, std~1.0215
```

× 損失関数

F.mse_loss(pred, target) 平均二乗誤差。差の二乗の平均。回帰に使用。

例:

```
import torch
import torch.nn.functional as F
pred = torch.tensor([1.0, 2.0, 3.0])
target = torch.tensor([1.5, 2.0, 2.5])
loss = F.mse_loss(pred, target)
print(f"Prediction: {pred}")
print(f"Target: {target}")
print(f"MSE Loss: {loss:.4f}")
```

実行結果:

```
Prediction: tensor([1., 2., 3.])
Target: tensor([1.5000, 2.0000, 2.5000])
MSE Loss: 0.1667
```

F.l1_loss(pred, target) 平均絶対誤差。*MSE* より外れ値に対してロバスト。

例:

```
import torch
import torch.nn.functional as F
pred = torch.tensor([1.0, 2.0, 3.0])
target = torch.tensor([1.5, 2.0, 2.5])
loss = F.l1_loss(pred, target)
print(f"Prediction: {pred}")
print(f"Target: {target}")
print(f"L1 Loss: {loss:.4f}")
```

実行結果:

```
Prediction: tensor([1., 2., 3.])
Target: tensor([1.5000, 2.0000, 2.5000])
L1 Loss: 0.3333
```

F.cross_entropy(logits, labels) *log_softmax* と *NLLLoss* を結合。多クラス分類の標準。

例:

```
import torch
import torch.nn.functional as F
logits = torch.tensor([[2.0, 0.5, 0.1], [0.1, 2.0, 0.5]])
labels = torch.tensor([0, 1])  # Class indices
loss = F.cross_entropy(logits, labels)
print(f"Logits: {logits}")
print(f"Labels: {labels}")
print(f"Cross Entropy Loss: {loss:.4f}")
```

実行結果:

```
Logits: tensor([[2.0000, 0.5000, 0.1000],
        [0.1000, 2.0000, 0.5000]])
Labels: tensor([0, 1])
Cross Entropy Loss: 0.3168
```

F.nll_loss(log_probs, labels) 負の対数尤度。*log_softmax* 出力と一緒に使用。

例:

```
import torch
import torch.nn.functional as F
log_probs = F.log_softmax(torch.tensor([[2.0, 0.5], [0.5, 2.0]]), dim=1)
labels = torch.tensor([0, 1])
loss = F.nll_loss(log_probs, labels)
print(f"NLL Loss: {loss:.4f}")
```

実行結果:

```
NLL Loss: 0.2014
```

F.binary_cross_entropy(pred, target) 二値交差エントロピー。*sigmoid* 出力の二値分類に使用。

例:

```
import torch
import torch.nn.functional as F
pred = torch.tensor([0.8, 0.4, 0.9])  # Probabilities
target = torch.tensor([1.0, 0.0, 1.0])  # Binary labels
loss = F.binary_cross_entropy(pred, target)
print(f"Pred: {pred}")
print(f"Target: {target}")
print(f"BCE Loss: {loss:.4f}")
```

実行結果:

```
Pred: tensor([0.8000, 0.4000, 0.9000])
Target: tensor([1., 0., 1.])
BCE Loss: 0.2798
```

F.kl_div(log_pred, target) *Kullback-Leibler* ダイバージェンス。分布間の差を測定。

例:

```
import torch
import torch.nn.functional as F
log_pred = F.log_softmax(torch.tensor([0.5, 0.3, 0.2]), dim=0)
target = F.softmax(torch.tensor([0.4, 0.4, 0.2]), dim=0)
loss = F.kl_div(log_pred, target, reduction='sum')
print(f"KL Divergence: {loss:.4f}")
```

実行結果:

```
KL Divergence: 0.0035
```

F.cosine_similarity(x1, x2, dim) ベクトル間の角度を測定。*1* = 同方向、*-1* = 反対方向。

例:

```
import torch
import torch.nn.functional as F
x1 = torch.tensor([[1.0, 0.0, 0.0]])
x2 = torch.tensor([[1.0, 1.0, 0.0]])
sim = F.cosine_similarity(x1, x2)
print(f"x1: {x1}")
```

```
print(f"x2: {x2}")
print(f"Cosine similarity: {sim}")
```

実行結果:

```
x1: tensor([[1., 0., 0.]])
x2: tensor([[1., 1., 0.]])
Cosine similarity: tensor([0.7071])
```

F.triplet_margin_loss(...) 類似項目は近く、異なる項目は遠くなるように埋め込みを学習。

例:

```
import torch
import torch.nn.functional as F
anchor = torch.randn(3, 128)
positive = anchor + 0.1 * torch.randn(3, 128)
negative = torch.randn(3, 128)
loss = F.triplet_margin_loss(anchor, positive, negative, margin=1.0)
print(f"Triplet loss: {loss:.4f}")
```

実行結果:

```
Triplet loss: 0.0000
```

□ プーリングと畳み込み

F.max_pool2d(x, kernel_size) 各ウィンドウの最大値を取ってダウンサンプリング。空間次元を削減。

例:

```
import torch
import torch.nn.functional as F
x = torch.arange(16).float().view(1, 1, 4, 4)
print("Input (4x4):")
print(x.squeeze())
result = F.max_pool2d(x, kernel_size=2)
print("MaxPool2d (kernel=2):")
print(result.squeeze())
```

実行結果:

```
Input (4x4):
tensor([[ 0.,  1.,  2.,  3.],
        [ 4.,  5.,  6.,  7.],
        [ 8.,  9., 10., 11.],
        [12., 13., 14., 15.]])
MaxPool2d (kernel=2):
tensor([[ 5.,  7.],
        [13., 15.]])
```

F.avg_pool2d(x, kernel_size) 各ウィンドウを平均してダウンサンプリング。*max pooling* より滑らか。

例:

```
import torch
import torch.nn.functional as F
x = torch.arange(16).float().view(1, 1, 4, 4)
print("Input (4x4):")
print(x.squeeze())
result = F.avg_pool2d(x, kernel_size=2)
print("AvgPool2d (kernel=2):")
print(result.squeeze())
```

実行結果:

```
Input (4x4):
tensor([[ 0.,  1.,  2.,  3.],
        [ 4.,  5.,  6.,  7.],
        [ 8.,  9., 10., 11.],
        [12., 13., 14., 15.]])
AvgPool2d (kernel=2):
tensor([[ 2.5000,  4.5000],
        [10.5000, 12.5000]])
```

`F.adaptive_max_pool2d(x, output_size)` 入力サイズに関係なく正確な出力サイズにプーリング。

例:

```
import torch
import torch.nn.functional as F
x = torch.randn(1, 1, 7, 7)
result = F.adaptive_max_pool2d(x, output_size=(2, 2))
print(f"Input: {x.shape} -> Output: {result.shape}")
```

実行結果:

```
Input: torch.Size([1, 1, 7, 7]) -> Output: torch.Size([1, 1, 2, 2])
```

`F.conv2d(x, weight, bias)` *2D* 畳み込み。*CNN* のコア操作。空間特徴を抽出。

例:

```
import torch
import torch.nn.functional as F
x = torch.randn(1, 1, 5, 5)  # batch, in_ch, H, W
weight = torch.randn(1, 1, 3, 3)  # out_ch, in_ch, kH, kW
result = F.conv2d(x, weight)
print(f"Input: {x.shape}")
print(f"Weight: {weight.shape}")
print(f"Output: {result.shape}")
```

実行結果:

```
Input: torch.Size([1, 1, 5, 5])
Weight: torch.Size([1, 1, 3, 3])
Output: torch.Size([1, 1, 3, 3])
```

`F.conv_transpose2d(x, weight)` 転置畳み込み（デコンボリューション）。空間次元をアップサンプリング。

例:

```
import torch
import torch.nn.functional as F
x = torch.randn(1, 1, 3, 3)
weight = torch.randn(1, 1, 3, 3)
result = F.conv_transpose2d(x, weight)
print(f"Input: {x.shape}")
print(f"Output: {result.shape}")
```

実行結果:

```
Input: torch.Size([1, 1, 3, 3])
Output: torch.Size([1, 1, 5, 5])
```

`F.interpolate(x, size, mode)` 補間を使ってテンソルサイズを調整。モード：*nearest*、*bilinear*、*bicubic*。

例:

```
import torch
import torch.nn.functional as F
x = torch.randn(1, 1, 4, 4)
result = F.interpolate(x, size=(8, 8), mode='bilinear', align_corners=False)
print(f"Input: {x.shape} -> Output: {result.shape}")
```

実行結果:

```
Input: torch.Size([1, 1, 4, 4]) -> Output: torch.Size([1, 1, 8, 8])
```

`F.pad(x, pad, mode)` テンソル周りにパディングを追加。畳み込みに使用。モード：*constant*、*reflect*、*replicate*。

例:

```
import torch
import torch.nn.functional as F
x = torch.arange(4).float().view(1, 1, 2, 2)
print("Input:")
print(x.squeeze())
padded = F.pad(x, (1, 1, 1, 1), mode='constant', value=0)
print("Padded (1 on each side):")
print(padded.squeeze())
```

実行結果:

```
Input:
tensor([[0., 1.],
        [2., 3.]])
Padded (1 on each side):
tensor([[0., 0., 0., 0.],
        [0., 0., 1., 0.],
        [0., 2., 3., 0.],
        [0., 0., 0., 0.]])
```

∞ 高度な操作

`torch.einsum('ij,jk->ik', a, b)` アインシュタインの縮約記法。多くのテンソル操作の柔軟な表記法。

例:

```
import torch
a = torch.randn(2, 3)
b = torch.randn(3, 4)
result = torch.einsum('ij,jk->ik', a, b)
print(f"einsum('ij,jk->ik'): {a.shape} x {b.shape} = {result.shape}")
# Verify it's matrix multiplication
print(f"Same as matmul: {torch.allclose(result, a @ b)}")
```

実行結果:

```
einsum('ij,jk->ik'): torch.Size([2, 3]) x torch.Size([3, 4]) = torch.Size([2, 4])
Same as matmul: True
```

`torch.topk(x, k, dim)  # top k values` k 個の最大値とインデックスを返す。完全ソートより高速。

例:

```
import torch
x = torch.tensor([1, 5, 3, 9, 2, 7])
vals, idxs = torch.topk(x, k=3)
print(f"Input: {x}")
print(f"Top 3 values: {vals}")
print(f"Top 3 indices: {idxs}")
```

実行結果:

```
Input: tensor([1, 5, 3, 9, 2, 7])
Top 3 values: tensor([9, 7, 5])
Top 3 indices: tensor([3, 5, 1])
```

`torch.sort(x, dim)  # sorted values` 次元に沿ってテンソルをソート。値と元のインデックスを返す。

例:

```
import torch
x = torch.tensor([3, 1, 4, 1, 5, 9])
vals, idxs = torch.sort(x)
print(f"Original: {x}")
print(f"Sorted: {vals}")
print(f"Indices: {idxs}")
```

実行結果:

```
Original: tensor([3, 1, 4, 1, 5, 9])
Sorted: tensor([1, 1, 3, 4, 5, 9])
Indices: tensor([1, 3, 0, 2, 4, 5])
```

`torch.argsort(x, dim)  # sort indices` テンソルをソートするインデックスを返す。

例:

```
import torch
x = torch.tensor([3, 1, 4, 1, 5])
idxs = torch.argsort(x)
print(f"Original: {x}")
print(f"Argsort: {idxs}")
print(f"Sorted via indices: {x[idxs]}")
```

実行結果:

```
Original: tensor([3, 1, 4, 1, 5])
Argsort: tensor([1, 3, 0, 2, 4])
Sorted via indices: tensor([1, 1, 3, 4, 5])
```

`torch.unique(x)  # unique values` 一意の要素を返す。オプションの *return_counts* で頻度取得。

例:

```
import torch
x = torch.tensor([1, 2, 2, 3, 1, 3, 3, 4])
unique = torch.unique(x)
print(f"Original: {x}")
print(f"Unique: {unique}")
```

実行結果:

```
Original: tensor([1, 2, 2, 3, 1, 3, 3, 4])
Unique: tensor([1, 2, 3, 4])
```

`torch.cat([t1, t2], dim)  # concat` 既存の次元に沿ってテンソルを連結。*dim* 以外の形状が同じ必要あり。

例:

```
import torch
a = torch.tensor([[1, 2], [3, 4]])
b = torch.tensor([[5, 6]])
result = torch.cat([a, b], dim=0)
print("Concatenated along dim 0:")
print(result)
```

実行結果:

```
Concatenated along dim 0:
tensor([[1, 2],
        [3, 4],
        [5, 6]])
```

`torch.stack([t1, t2], dim)  # new dim` 新しい次元に沿ってテンソルをスタック。すべてのテンソルが同じ形状必要。

例:

```
import torch
a = torch.tensor([1, 2, 3])
b = torch.tensor([4, 5, 6])
result = torch.stack([a, b], dim=0)
print("Stacked (creates new dim):")
```

```
print(result)
print(f"Shape: {result.shape}")
```

実行結果:

```
Stacked (creates new dim):
tensor([[1, 2, 3],
        [4, 5, 6]])
Shape: torch.Size([2, 3])
```

torch.split(x, size, dim) # split テンソルを指定サイズのチャンクに分割。最後のチャンクは小さい場合あり。

例:

```
import torch
x = torch.arange(10)
splits = torch.split(x, 3)
print(f"Original: {x}")
print("Split into chunks of 3:")
for i, s in enumerate(splits):
    print(f"  {i}: {s}")
```

実行結果:

```
Original: tensor([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
Split into chunks of 3:
  0: tensor([0, 1, 2])
  1: tensor([3, 4, 5])
  2: tensor([6, 7, 8])
  3: tensor([9])
```

torch.chunk(x, chunks, dim) # chunks テンソルを指定数のチャンクに分割。

例:

```
import torch
x = torch.arange(12)
chunks = torch.chunk(x, 4)
print(f"Original: {x}")
print("Split into 4 chunks:")
for i, c in enumerate(chunks):
    print(f"  {i}: {c}")
```

実行結果:

```
Original: tensor([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11])
Split into 4 chunks:
  0: tensor([0, 1, 2])
  1: tensor([3, 4, 5])
  2: tensor([6, 7, 8])
  3: tensor([ 9, 10, 11])
```

torch.broadcast_to(x, shape) テンソルを明示的に新しい形状にブロードキャスト。データをコピーしない。

例:

```
import torch
x = torch.tensor([1, 2, 3])
result = torch.broadcast_to(x, (3, 3))
print(f"Original: {x}")
print("Broadcast to (3, 3):")
print(result)
```

実行結果:

```
Original: tensor([1, 2, 3])
Broadcast to (3, 3):
tensor([[1, 2, 3],
        [1, 2, 3],
        [1, 2, 3]])
```

`torch.flatten(x, start, end)` テンソルを平坦化。部分平坦化用のオプション開始/終了次元あり。

例:

```
import torch
x = torch.randn(2, 3, 4)
flat = torch.flatten(x)
partial = torch.flatten(x, start_dim=1)
print(f"Original: {x.shape}")
print(f"Fully flat: {flat.shape}")
print(f"Flatten from dim 1: {partial.shape}")
```

実行結果:

```
Original: torch.Size([2, 3, 4])
Fully flat: torch.Size([24])
Flatten from dim 1: torch.Size([2, 12])
```

∂ 自動微分

`x.requires_grad_(True)` テンソルの勾配追跡を有効化。逆伝播に必要。

例:

```
import torch
x = torch.tensor([1.0, 2.0, 3.0])
print(f"Before: requires_grad = {x.requires_grad}")
x.requires_grad_(True)
print(f"After: requires_grad = {x.requires_grad}")
```

実行結果:

```
Before: requires_grad = False
After: requires_grad = True
```

`y.backward()` 逆伝播で勾配を計算。*.grad* 属性に格納。

例:

```
import torch
x = torch.tensor([2.0, 3.0], requires_grad=True)
```

```
y = (x ** 2).sum()  # y = x1^2 + x2^2
y.backward()
print(f"x = {x}")
print(f"y = x^2.sum() = {y}")
print(f"dy/dx = {x.grad}")  # 2*x
```

実行結果:

```
x = tensor([2., 3.], requires_grad=True)
y = x^2.sum() = 13.0
dy/dx = tensor([4., 6.])
```

x.grad # gradient *backward()* 後に累積された勾配を格納。*zero_grad()* でリセット。

例:

```
import torch
x = torch.tensor(3.0, requires_grad=True)
y = x ** 2
y.backward()
print(f"x = {x}")
print(f"y = x^2 = {y}")
print(f"dy/dx = {x.grad}")
```

実行結果:

```
x = 3.0
y = x^2 = 9.0
dy/dx = 6.0
```

x.detach() 計算グラフから分離されたテンソルを返す。勾配の流れを停止。

例:

```
import torch
x = torch.tensor([1.0, 2.0], requires_grad=True)
y = x * 2
z = y.detach()
print(f"y requires_grad: {y.requires_grad}")
print(f"z requires_grad: {z.requires_grad}")
```

実行結果:

```
y requires_grad: True
z requires_grad: False
```

x.clone() 同じデータのコピーを作成。変更しても元に影響しない。

例:

```
import torch
x = torch.tensor([1.0, 2.0])
y = x.clone()
y[0] = 99
print(f"Original x: {x}")
print(f"Cloned y: {y}")
```

実行結果:

```
Original x: tensor([1., 2.])
Cloned y: tensor([99.,  2.])
```

`with torch.no_grad():` 勾配計算を無効化するコンテキストマネージャー。推論用。

例:

```
import torch
x = torch.tensor([1.0, 2.0], requires_grad=True)
with torch.no_grad():
    y = x * 2
    print(f"Inside no_grad: y.requires_grad = {y.requires_grad}")
z = x * 2
print(f"Outside: z.requires_grad = {z.requires_grad}")
```

実行結果:

```
Inside no_grad: y.requires_grad = False
Outside: z.requires_grad = True
```

`torch.autograd.grad(y, x)` *.grad* を変更せずに勾配を計算。二階微分に便利。

例:

```
import torch
x = torch.tensor([1.0, 2.0], requires_grad=True)
y = (x ** 3).sum()  # y = x1^3 + x2^3
grad = torch.autograd.grad(y, x)
print(f"x = {x}")
print(f"dy/dx = {grad[0]}")  # 3*x^2
```

実行結果:

```
x = tensor([1., 2.], requires_grad=True)
dy/dx = tensor([ 3., 12.])
```

`optimizer.zero_grad()` 勾配を *0* にリセット。各 *backward pass* 前に呼び出す。

例:

```
import torch
import torch.optim as optim
x = torch.tensor([1.0], requires_grad=True)
optimizer = optim.SGD([x], lr=0.1)
y = x ** 2
y.backward()
print(f"Grad before zero_grad: {x.grad}")
optimizer.zero_grad()
print(f"Grad after zero_grad: {x.grad}")
```

実行結果:

```
Grad before zero_grad: tensor([2.])
Grad after zero_grad: None
```

`optimizer.step()` 計算された勾配でパラメータを更新。$x = x - lr$ grad。*

例:

```
import torch
import torch.optim as optim
x = torch.tensor([5.0], requires_grad=True)
optimizer = optim.SGD([x], lr=0.1)
print(f"Before: x = {x.item():.4f}")
y = x ** 2
y.backward()
optimizer.step()
print(f"After step: x = {x.item():.4f}")
```

実行結果:

```
Before: x = 5.0000
After step: x = 4.0000
```

torch.nn.utils.clip_grad_norm_(params, max) 勾配爆発を防ぐため勾配ノルムをクリップ。*RNN* に必須。

例:

```
import torch
import torch.nn as nn
model = nn.Linear(10, 5)
x = torch.randn(1, 10)
loss = model(x).sum()
loss.backward()
norm_before = sum(p.grad.norm()**2 for p in model.parameters())**0.5
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
norm_after = sum(p.grad.norm()**2 for p in model.parameters())**0.5
print(f"Grad norm before: {norm_before:.4f}")
print(f"Grad norm after clip: {norm_after:.4f}")
```

実行結果:

```
Grad norm before: 6.8840
Grad norm after clip: 1.0000
```

🖥 デバイス操作

torch.cuda.is_available() *CUDA GPU* が利用可能か確認。デバイス選択ロジックに使用。

例:

```
import torch
print(f"CUDA available: {torch.cuda.is_available()}")
print(f"MPS available: {torch.backends.mps.is_available()}")
```

実行結果:

```
CUDA available: False
MPS available: True
```

torch.device('cuda'/'cpu'/'mps') デバイスオブジェクトを作成。*.to(device)* とともにデバイス配置に使用。

例:

```python
import torch
cpu = torch.device('cpu')
print(f"CPU device: {cpu}")
if torch.cuda.is_available():
    gpu = torch.device('cuda')
    print(f"GPU device: {gpu}")
elif torch.backends.mps.is_available():
    mps = torch.device('mps')
    print(f"MPS device: {mps}")
```

実行結果:

```
CPU device: cpu
MPS device: mps
```

`x.to(device)  # move to device` テンソルを指定デバイスに移動。*GPU* 訓練に必須。

例:

```python
import torch
x = torch.tensor([1.0, 2.0])
device = torch.device('cpu')
x_dev = x.to(device)
print(f"Tensor on: {x_dev.device}")
```

実行結果:

```
Tensor on: cpu
```

`x.to(dtype)  # change dtype` テンソルを異なるデータ型に変換。

例:

```python
import torch
x = torch.tensor([1, 2, 3])
print(f"Original dtype: {x.dtype}")
x_float = x.to(torch.float32)
print(f"After to(float32): {x_float.dtype}")
```

実行結果:

```
Original dtype: torch.int64
After to(float32): torch.float32
```

`x.cuda(), x.cpu()  # shortcuts` *CPU* と *CUDA* 間の移動のショートカットメソッド。

例:

```python
import torch
x = torch.tensor([1.0, 2.0])
print(f"Original device: {x.device}")
x_cpu = x.cpu()
print(f"After cpu(): {x_cpu.device}")
```

実行結果:

```
Original device: cpu
After cpu(): cpu
```

`x.device  # check current device` テンソルが格納されているデバイスを返す。

例:

```
import torch
x = torch.tensor([1.0, 2.0])
print(f"Device: {x.device}")
print(f"Device type: {x.device.type}")
```

実行結果:

```
Device: cpu
Device type: cpu
```

`x.is_cuda  # boolean check` テンソルが *CUDA* デバイス上にあれば *True* を返す。

例:

```
import torch
x = torch.tensor([1.0, 2.0])
print(f"is_cuda: {x.is_cuda}")
```

実行結果:

```
is_cuda: False
```

`torch.cuda.empty_cache()` 未使用のキャッシュ *GPU* メモリを解放。*OOM* エラーに有効。

例:

```
import torch
# Free unused cached memory (only affects CUDA)
if torch.cuda.is_available():
    torch.cuda.empty_cache()
    print("CUDA cache cleared")
else:
    print("CUDA not available - cache clearing skipped")
```

実行結果:

```
CUDA not available - cache clearing skipped
```

`torch.cuda.device_count()` 利用可能な *CUDA* デバイス数を返す。

例:

```
import torch
if torch.cuda.is_available():
    count = torch.cuda.device_count()
    print(f"Number of GPUs: {count}")
else:
    print("CUDA not available")
```

実行結果:

```
CUDA not available
```

※ ユーティリティ

x.dtype, x.shape, x.size() 基本テンソル属性。*shape* と *size()* は同じ。

例:

```
import torch
x = torch.randn(2, 3, 4)
print(f"dtype: {x.dtype}")
print(f"shape: {x.shape}")
print(f"size(): {x.size()}")
```

実行結果:

```
dtype: torch.float32
shape: torch.Size([2, 3, 4])
size(): torch.Size([2, 3, 4])
```

x.numel() # number of elements 要素の総数を返す。すべての次元の積。

例:

```
import torch
x = torch.randn(2, 3, 4)
print(f"Shape: {x.shape}")
print(f"Number of elements: {x.numel()}")
```

実行結果:

```
Shape: torch.Size([2, 3, 4])
Number of elements: 24
```

x.dim() # number of dimensions テンソルの次元数 (ランク) を返す。

例:

```
import torch
x = torch.randn(2, 3, 4)
print(f"Shape: {x.shape}")
print(f"Dimensions: {x.dim()}")
```

実行結果:

```
Shape: torch.Size([2, 3, 4])
Dimensions: 3
```

x.ndimension() # same as dim() *dim()* のエイリアス。次元数を返す。

例:

```
import torch
x = torch.randn(2, 3, 4)
print(f"ndimension(): {x.ndimension()}")
print(f"Same as dim(): {x.dim()}")
```

実行結果:

```
ndimension(): 3
Same as dim(): 3
```

x.is_contiguous() テンソルがメモリ上で連続しているか確認。*view()* に必要。

例:

```
import torch
x = torch.randn(3, 4)
y = x.transpose(0, 1)
print(f"Original is_contiguous: {x.is_contiguous()}")
print(f"Transposed is_contiguous: {y.is_contiguous()}")
```

実行結果:

```
Original is_contiguous: True
Transposed is_contiguous: False
```

x.float(), x.int(), x.long() *dtype* 変換のショートカットメソッド。

例:

```
import torch
x = torch.tensor([1, 2, 3])
print(f"Original: {x.dtype}")
print(f"float(): {x.float().dtype}")
print(f"long(): {x.long().dtype}")
```

実行結果:

```
Original: torch.int64
float(): torch.float32
long(): torch.int64
```

x.half(), x.double() # fp16, fp64 半精度 (*16* ビット) は高速訓練用、*double* は高精度用。

例:

```
import torch
x = torch.tensor([1.0, 2.0])
print(f"Original: {x.dtype}")
print(f"half() (fp16): {x.half().dtype}")
print(f"double() (fp64): {x.double().dtype}")
```

実行結果:

```
Original: torch.float32
half() (fp16): torch.float16
double() (fp64): torch.float64
```

torch.from_numpy(arr) *NumPy* 配列をテンソルに変換。元の配列とメモリを共有。

例:

```
import torch
import numpy as np
arr = np.array([1, 2, 3])
x = torch.from_numpy(arr)
print(f"NumPy: {arr}, dtype={arr.dtype}")
print(f"Tensor: {x}, dtype={x.dtype}")
```

実行結果:

```
NumPy: [1 2 3], dtype=int64
Tensor: tensor([1, 2, 3]), dtype=torch.int64
```

`x.numpy()  # CPU only` テンソルを *NumPy* 配列に変換。*CPU* 上にある必要あり。

例:

```
import torch
x = torch.tensor([1.0, 2.0, 3.0])
arr = x.numpy()
print(f"Tensor: {x}")
print(f"NumPy: {arr}, dtype={arr.dtype}")
```

実行結果:

```
Tensor: tensor([1., 2., 3.])
NumPy: [1. 2. 3.], dtype=float32
```

`torch.save(obj, path)` テンソルまたはモデルをファイルに保存。*pickle* 直列化を使用。

例:

```
import torch
import os
x = torch.tensor([1, 2, 3])
torch.save(x, '/tmp/tensor.pt')
print(f"Saved tensor to /tmp/tensor.pt")
print(f"File size: {os.path.getsize('/tmp/tensor.pt')} bytes")
```

実行結果:

```
Saved tensor to /tmp/tensor.pt
File size: 1570 bytes
```

`torch.load(path)` 保存されたテンソルまたはモデルをファイルから読み込む。

例:

```
import torch
torch.save(torch.tensor([1, 2, 3]), '/tmp/tensor.pt')
loaded = torch.load('/tmp/tensor.pt', weights_only=True)
print(f"Loaded: {loaded}")
```

実行結果:

```
Loaded: tensor([1, 2, 3])
```

`torch.manual_seed(seed)` 再現性のためランダムシードを設定。

例:

```
import torch
torch.manual_seed(42)
a = torch.rand(3)
torch.manual_seed(42)
b = torch.rand(3)
print(f"First: {a}")
print(f"Second (same seed): {b}")
print(f"Equal: {torch.equal(a, b)}")
```

実行結果:

```
First: tensor([0.8823, 0.9150, 0.3829])
Second (same seed): tensor([0.8823, 0.9150, 0.3829])
Equal: True
```

≈ 比較操作

torch.eq(a, b) or a == b 要素ごとの等価比較。ブールテンソルを返す。

例:

```
import torch
a = torch.tensor([1, 2, 3])
b = torch.tensor([1, 0, 3])
print(f"a: {a}")
print(f"b: {b}")
print(f"a == b: {a == b}")
```

実行結果:

```
a: tensor([1, 2, 3])
b: tensor([1, 0, 3])
a == b: tensor([ True, False,  True])
```

torch.ne(a, b) or a != b 要素ごとの不等比較。

例:

```
import torch
a = torch.tensor([1, 2, 3])
b = torch.tensor([1, 0, 3])
print(f"a != b: {a != b}")
```

実行結果:

```
a != b: tensor([False,  True, False])
```

torch.gt(a, b) or a > b 要素ごとの大なり比較。

例:

```
import torch
a = torch.tensor([1, 2, 3])
b = torch.tensor([2, 2, 2])
print(f"a: {a}, b: {b}")
print(f"a > b: {a > b}")
```

実行結果:

```
a: tensor([1, 2, 3]), b: tensor([2, 2, 2])
a > b: tensor([False, False,  True])
```

torch.lt(a, b) or a < b 要素ごとの小なり比較。

例:

```
import torch
a = torch.tensor([1, 2, 3])
b = torch.tensor([2, 2, 2])
print(f"a < b: {a < b}")
```

実行結果:

```
a < b: tensor([ True, False, False])
```

torch.ge(a, b) or a >= b 要素ごとの以上比較。

例:

```
import torch
a = torch.tensor([1, 2, 3])
b = torch.tensor([2, 2, 2])
print(f"a >= b: {a >= b}")
```

実行結果:

```
a >= b: tensor([False,  True,  True])
```

torch.le(a, b) or a <= b 要素ごとの以下比較。

例:

```
import torch
a = torch.tensor([1, 2, 3])
b = torch.tensor([2, 2, 2])
print(f"a <= b: {a <= b}")
```

実行結果:

```
a <= b: tensor([ True,  True, False])
```

torch.allclose(a, b, rtol, atol) すべての要素が許容範囲内で近いか確認。*float* 比較用。

例:

```
import torch
a = torch.tensor([1.0, 2.0])
b = torch.tensor([1.0001, 2.0001])
print(f"a: {a}")
print(f"b: {b}")
print(f"allclose (default tol): {torch.allclose(a, b)}")
print(f"allclose (rtol=1e-3): {torch.allclose(a, b, rtol=1e-3)}")
```

実行結果:

```
a: tensor([1., 2.])
b: tensor([1.0001, 2.0001])
allclose (default tol): False
allclose (rtol=1e-3): True
```

torch.isnan(x) 要素が *NaN*（非数）の場所で *True* を返す。

例:

```
import torch
x = torch.tensor([1.0, float('nan'), 3.0])
print(f"x: {x}")
print(f"isnan: {torch.isnan(x)}")
```

実行結果:

```
x: tensor([1., nan, 3.])
isnan: tensor([False,  True, False])
```

torch.isinf(x)　要素が無限大の場所で *True* を返す。

例:

```
import torch
x = torch.tensor([1.0, float('inf'), float('-inf')])
print(f"x: {x}")
print(f"isinf: {torch.isinf(x)}")
```

実行結果:

```
x: tensor([1., inf, -inf])
isinf: tensor([False,  True,  True])
```

torch.isfinite(x)　要素が有限 (*inf* でなく *nan* でない) の場所で *True* を返す。

例:

```
import torch
x = torch.tensor([1.0, float('inf'), float('nan')])
print(f"x: {x}")
print(f"isfinite: {torch.isfinite(x)}")
```

実行結果:

```
x: tensor([1., inf, nan])
isfinite: tensor([ True, False, False])
```

● テンソルメソッド

x.T　# transpose (2D)　*2D* 転置の省略形。高次元には *permute* を使用。

例:

```
import torch
x = torch.tensor([[1, 2, 3], [4, 5, 6]])
print("Original:")
print(x)
print("x.T:")
print(x.T)
```

実行結果:

```
Original:
tensor([[1, 2, 3],
        [4, 5, 6]])
x.T:
```

```
tensor([[1, 4],
        [2, 5],
        [3, 6]])
```

x.H # conjugate transpose 複素テンソルの共役転置。転置 + 共役。

例:

```
import torch
x = torch.tensor([[1+2j, 3+4j], [5+6j, 7+8j]])
print("Original:")
print(x)
print("x.H (conjugate transpose):")
print(x.H)
```

実行結果:

```
Original:
tensor([[1.+2.j, 3.+4.j],
        [5.+6.j, 7.+8.j]])
x.H (conjugate transpose):
tensor([[1.-2.j, 5.-6.j],
        [3.-4.j, 7.-8.j]])
```

x.real, x.imag # complex parts 複素テンソルの実部と虚部にアクセス。

例:

```
import torch
x = torch.tensor([1+2j, 3+4j])
print(f"Complex: {x}")
print(f"Real: {x.real}")
print(f"Imag: {x.imag}")
```

実行結果:

```
Complex: tensor([1.+2.j, 3.+4.j])
Real: tensor([1., 3.])
Imag: tensor([2., 4.])
```

x.abs(), x.neg() # absolute, negate 絶対値と符号反転のメソッド。

例:

```
import torch
x = torch.tensor([-3, -1, 0, 2, 4])
print(f"x: {x}")
print(f"abs: {x.abs()}")
print(f"neg: {x.neg()}")
```

実行結果:

```
x: tensor([-3, -1,  0,  2,  4])
abs: tensor([3, 1, 0, 2, 4])
neg: tensor([ 3,  1,  0, -2, -4])
```

x.reciprocal(), x.pow(n) メソッドとしての逆数（*1/x*）と累乗演算。

例:

```
import torch
x = torch.tensor([1.0, 2.0, 4.0])
print(f"x: {x}")
print(f"reciprocal: {x.reciprocal()}")
print(f"pow(2): {x.pow(2)}")
```

実行結果:

```
x: tensor([1., 2., 4.])
reciprocal: tensor([1.0000, 0.5000, 0.2500])
pow(2): tensor([ 1.,  4., 16.])
```

x.sqrt(), x.exp(), x.log() テンソルメソッドとしての一般的な数学演算。

例:

```
import torch
x = torch.tensor([1.0, 4.0, 9.0])
print(f"x: {x}")
print(f"sqrt: {x.sqrt()}")
print(f"exp: {torch.tensor([0.0, 1.0]).exp()}")
print(f"log: {torch.tensor([1.0, 2.718]).log()}")
```

実行結果:

```
x: tensor([1., 4., 9.])
sqrt: tensor([1., 2., 3.])
exp: tensor([1.0000, 2.7183])
log: tensor([0.0000, 0.9999])
```

x.item() # get scalar value 単一要素テンソルから *Python* 数値としてスカラー値を抽出。

例:

```
import torch
x = torch.tensor(3.14159)
val = x.item()
print(f"Tensor: {x}")
print(f"Python float: {val}")
print(f"Type: {type(val)}")
```

実行結果:

```
Tensor: 3.141590118408203
Python float: 3.141590118408203
Type: <class 'float'>
```

x.tolist() # to Python list テンソルをネストされた *Python* リストに変換。

例:

```
import torch
x = torch.tensor([[1, 2], [3, 4]])
lst = x.tolist()
print(f"Tensor: {x}")
```

```
print(f"List: {lst}")
print(f"Type: {type(lst)}")
```

実行結果:

```
Tensor: tensor([[1, 2],
        [3, 4]])
List: [[1, 2], [3, 4]]
Type: <class 'list'>
```

x.all(), x.any() # boolean checks すべてまたはいずれかの要素が *True* か確認。

例:

```
import torch
x = torch.tensor([True, True, False])
print(f"x: {x}")
print(f"all: {x.all()}")
print(f"any: {x.any()}")
```

実行結果:

```
x: tensor([ True,  True, False])
all: False
any: True
```

x.nonzero() # non-zero indices 非ゼロ要素のインデックスを返す。

例:

```
import torch
x = torch.tensor([0, 1, 0, 2, 0, 3])
indices = x.nonzero()
print(f"x: {x}")
print(f"Non-zero indices: {indices.squeeze()}")
```

実行結果:

```
x: tensor([0, 1, 0, 2, 0, 3])
Non-zero indices: tensor([1, 3, 5])
```

x.fill_(val), x.zero_() # in-place 値またはゼロでインプレース充填。アンダースコア接尾辞 = インプレース。

例:

```
import torch
x = torch.empty(2, 3)
x.fill_(5.0)
print("After fill_(5.0):")
print(x)
x.zero_()
print("After zero_():")
print(x)
```

実行結果:

```
After fill_(5.0):
tensor([[5., 5., 5.],
        [5., 5., 5.]])
After zero_():
tensor([[0., 0., 0.],
        [0., 0., 0.]])
```

x.normal_(), x.uniform_() # random インプレースランダム初期化。重み初期化に便利。

例:

```
import torch
torch.manual_seed(42)
x = torch.empty(2, 3)
x.normal_(mean=0, std=1)
print("After normal_(0, 1):")
print(x)
x.uniform_(0, 1)
print("After uniform_(0, 1):")
print(x)
```

実行結果:

```
After normal_(0, 1):
tensor([[ 0.3367,  0.1288,  0.2345],
        [ 0.2303, -1.1229, -0.1863]])
After uniform_(0, 1):
tensor([[0.8694, 0.5677, 0.7411],
        [0.4294, 0.8854, 0.5739]])
```

x.add_(y), x.mul_(y) # in-place ops インプレース算術。テンソルを直接変更してメモリを節約。

例:

```
import torch
x = torch.tensor([1.0, 2.0, 3.0])
print(f"Original: {x}")
x.add_(10)
print(f"After add_(10): {x}")
x.mul_(2)
print(f"After mul_(2): {x}")
```

実行結果:

```
Original: tensor([1., 2., 3.])
After add_(10): tensor([11., 12., 13.])
After mul_(2): tensor([22., 24., 26.])
```